

Study Reference: Relative Positional Encodings in Transformers

Deep Learning Reference Notes

May 29, 2026

Contents

1	Introduction & Core Philosophy	2
2	The Mechanism: Incorporation with Self-Attention	2
2.1	Mathematical Formulation	2
2.2	Clipping (Maximum Distance Cap)	2
3	Numerical Walkthrough Example	2
3.1	Step 1: Content Score Computation	3
3.2	Step 2: Distance and Table Lookup	3
3.3	Step 3: Score Integration	3
3.4	Invariance to Absolute Shifts	3
4	Nature of the Bias: A Learnable Parameter	3
5	Engineering Trade-offs & Evolution	4
5.1	Architectural Challenges	4
5.2	Modern Paradigms: Rotary Position Embeddings (RoPE)	5

1 Introduction & Core Philosophy

In standard Transformer architectures, sequential data is processed in parallel rather than token-by-token. Consequently, the network is inherently invariant to sequence order. To inject positional information, the model requires positional encodings.

- **Absolute Positional Encoding:** Assigns a unique coordinate or static vector to each absolute position index $(1, 2, 3, \dots, N)$. A major limitation is poor extrapolation: the model cannot natively handle sequence lengths longer than those seen during training.
- **Relative Positional Encoding:** Focuses entirely on the pairwise distance (the gap) between tokens rather than their absolute index in the sequence. It mimics human reading logic, where the directional distance between a pronoun and its antecedent matters more than their exact page coordinates.

2 The Mechanism: Incorporation with Self-Attention

In standard scaled dot-product self-attention, the attention score between a Query (Q) and a Key (K) is computed purely based on semantic content:

$$\text{Attention Score} = QK^T \tag{1}$$

When incorporating relative positional encodings via a **Relative Bias** mechanism, a distance-dependent trainable parameter is added directly to this logit calculation.

2.1 Mathematical Formulation

Let i be the index of the Query token and j be the index of the Key token. The relative distance is defined as:

$$d = j - i \tag{2}$$

The modified attention score A_{ij} between token i and token j becomes:

$$A_{ij} = (Q_i K_j^T) + B_{(j-i)} \tag{3}$$

Where $B_{(j-i)}$ is the relative positional bias corresponding to the directed distance d .

2.2 Clipping (Maximum Distance Cap)

To limit computational complexity and prevent overfitting on rare, extremely long distances, models often employ a clipping function $h(d)$ with a maximum threshold k :

$$h(d) = \max(-k, \min(k, d)) \tag{4}$$

Distances beyond the window $[-k, k]$ are mapped to the same edge-case bias parameters.

3 Numerical Walkthrough Example

Consider processing the 3-token phrase: **"Go eat food"**. We compute the attention logic for the token **"Go"** ($i = 0$) attending to **"food"** ($j = 2$).

3.1 Step 1: Content Score Computation

The semantic similarity based on token representations yields a raw score:

$$Q_{\text{Go}} \cdot K_{\text{food}}^T = 4.5$$

3.2 Step 2: Distance and Table Lookup

The relative distance is:

$$d = 2 - 0 = +2$$

The model queries its learned lookup table for $B_{(+2)}$. Assume the trained parameters in the network’s lookup table are configured as follows:

Table 1: Learned Relative Position Bias Table (Hypothetical Values)

Distance (d)	Bias Value (B_d)	Interpretation
-2	-1.5	Two tokens to the left (Weakened connection)
-1	+0.5	One token to the left (Moderate connection)
0	+2.0	Identity self-attention (Strong connection)
+1	+1.2	One token to the right (Strong connection)
+2	-0.8	Two tokens to the right (Attenuated connection)

3.3 Step 3: Score Integration

The final attention score is calculated by adding the content score and the retrieved relative bias:

$$A_{\text{Go,food}} = 4.5 + (-0.8) = 3.7 \quad (5)$$

3.4 Invariance to Absolute Shifts

If the sequence shifts to **"Please Go eat food now"**, the indices become:

- $\text{Index}(\text{"Go"}) = 1$
- $\text{Index}(\text{"food"}) = 3$

The relative distance remains identical: $d = 3 - 1 = +2$. Therefore, the model retrieves the exact same bias parameter ($B_{(+2)} = -0.8$), proving translation invariance.

4 Nature of the Bias: A Learnable Parameter

A crucial aspect of this architecture is that the bias values (B_d) are ****learnable parameters**** (weights), not static mathematical constants or manual heuristics.

1. **Initialization:** The bias table is initialized with small, random weights at the beginning of training.
2. **Forward Pass:** The random values are used to compute attention matrices and generate downstream sequence predictions.
3. **Backpropagation:** The prediction error (Loss) is evaluated. Using ****Gradient Descent****, the network backpropagates gradients back to the relative position table.

4. **Optimization:** Over billions of tokens, the values optimize to fit natural language syntax (e.g., boosting values for close-range syntactic relationships and dampening long-distance noise).

—

5 Engineering Trade-offs & Evolution

5.1 Architectural Challenges

1. Computational Resource Hog

With absolute positioning, you stamp the word positions once at the very beginning of the network, and the initial layout is complete. With relative positioning, however, the model must dynamically calculate the distance between every single pair of tokens during the self-attention mechanism.

For instance:

- Given a short sentence of 10 words, there are $10 \times 10 = 100$ unique pairs to evaluate.
- Given a comprehensive document of 10,000 words, the matrix scales quadratically to $10,000 \times 10,000 = 100,000,000$ distinct pairs to analyze.

Because this calculation is tightly coupled with the core mathematical transformations inside every attention head, it demands significantly higher memory bandwidth and GPU processing power, ultimately bottlenecking training throughput and inference speeds.

2. “Distance” Does Not Equal “Meaning”

Relative encoding operates on the fundamental assumption that tokens spatially close to one another are inherently highly correlated, whereas tokens separated by large gaps are less related. While this heuristic holds broadly true, natural human language introduces significant structural variance.

Consider the following linguistic example:

*“The **dog** that was chased by the two very large, angry cats yesterday morning **ran** up the tree.”*

The structural core of the sentence—the subject (**dog**) and its corresponding verb (**ran**)—are separated by a dense sequence of modifier tokens. Because relative encoding schemes typically “clip” or downplay distances after a predefined numerical threshold, the network can struggle to map these crucial, long-distance syntactic dependencies as robustly as an absolute system that explicitly maintains the structural global index of the page.

3. Loss of Macro-Context (Absolute Global Picture)

In many document processing scenarios, the absolute geographical coordinate of a token within a file carries deterministic semantic weight:

- In a legal contract or statute, the opening lines strictly define the binding parties, jurisdictional metadata, and title.
- In literature, the foundational paragraphs establish baseline narrative framing and genre context.

Under a pure relative positional encoding regime, the network recognizes localized sequence lengths and local relational distances, but it remains fundamentally blind to whether a given paragraph resides at the absolute beginning, the true geometric center, or the final concluding appendix of a massive text block. It inherently sacrifices that global anchoring property.

4. Hardware and Algorithmic Complexity

Modern highly parallelized accelerators (such as Tensor Processing Units and Graphical Processing Units) are fundamentally optimized to execute static, highly repetitive matrix arithmetic at massive scales. Absolute position embeddings fit cleanly into this paradigm; they exist as a static lookup matrix added to the input layer at lightning speed.

Conversely, relative position implementations force the network to construct, manipulate, and cache dynamic, shifting distance bias matrices across individual token lengths. Designing custom CUDA kernels or highly optimized matrix execution steps for these architectures introduces significant algorithmic complexity, making optimization for consumer-grade production hardware highly challenging.

5.2 Modern Paradigms: Rotary Position Embeddings (RoPE)

To circumvent the overhead of storing and querying large relative bias lookup tables, state-of-the-art models (such as Llama and Mistral) utilize **Rotary Position Embeddings (RoPE)**.

Instead of adding an external scalar bias parameter, RoPE applies a vector rotation to the Query and Key states in complex vector space based on their absolute positions. When the dot product $Q_i K_j^T$ is calculated, the inner product naturally preserves the relative distance angle $(j - i)$, merging the computational efficiency of absolute indexing with the generalization power of relative positioning.